

ГУАП

КАФЕДРА № 14

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

Старший преподаватель	14.05.2026	Н.И. Синёв
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №5

Поразрядная обработка

по курсу: Программирование на языках Ассемблера

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	2026	14.05.2026	А.Н. Корякин
1443		подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

Описание задачи

Написать программу с поразрядной обработкой в ассемблере Apple ARM 64.

Формализация

Все вычисления, проверки и преобразования должны быть реализованы через пользовательскую функцию. Заменить каждый байт X на количество единиц в нём. Результат программы должен выглядеть следующим образом, результат - число после преобразований. Задача принимает одно значение X - длинное целое число, вводимое пользователем с клавиатуры. Ссылка на все исходники на GitHub: <https://github.com/Snctnm/Assembler>

Блок-схема

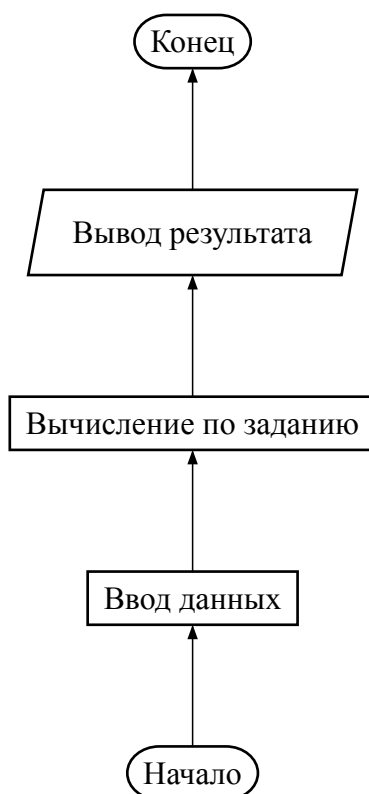


Рис. 1. Блок-схема программы

Исходный код программы

Код на ассемблере:

```
// Лабораторная 5. Поразрядная обработка. Вариант 54.  
// Заменить каждый байт X на количество единиц в нём.  
// Все вычисления в пользовательской функции transform_porcount.  
  
.global _main  
  
.text  
.align 2  
  
_main:  
    // Выравнивание стека и сохранение регистров кадра
```

```

sub sp, sp, #112
stp x29, x30, [sp, #96]
add x29, sp, #96

// --- Ввод X ---
adrp x0, prompt@PAGE
add x0, x0, prompt@PAGEOFF
mov x8, #0
bl _printf

adrp x0, fmt_in@PAGE
add x0, x0, fmt_in@PAGEOFF
add x1, x29, #-8           // адрес локальной переменной для ввода
str x1, [sp]              // Apple ABI: дублирование аргумента в стек для
variadic
mov x8, #0
bl _scanf

// Вызов пользовательской функции
ldr x0, [x29, #-8]        // загрузка X в x0
bl transform_popcount     // результат вернётся в x0

str x0, [x29, #-16]       // сохранение результата

// Вывод результата
adrp x0, fmt_out@PAGE
add x0, x0, fmt_out@PAGEOFF
ldr x1, [x29, #-16]
str x1, [sp]              // Apple ABI: дублирование аргумента в стек для
variadic
mov x8, #0
bl _printf

// Выход
ldp x29, x30, [sp, #96]
add sp, sp, #112
mov x0, #0
ret

/* Функция: transform_popcount
   ВХОД:  x0 = X (длинное целое число)
   ВЫХОД: x0 = результат (каждый байт заменён на popcount)
*/
transform_popcount:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    mov x9, #0             // аккумулятор результата
    mov x10, #0            // текущий сдвиг (0, 8, 16, ..., 56)
    mov x11, #8            // счётчик байтов

```

```

byte_loop:
    // 1. Извлечение текущего байта из X
    lsr x12, x0, x10
    and x12, x12, #0xFF

    // 2. Поразрядный подсчёт единиц в байте
    mov x13, #0           // счётчик установленных битов
    mov x14, x12          // рабочая копия байта
    mov x15, #8           // цикл на 8 бит

bit_loop:
    and x16, x14, #1      // выделяем младший бит
    add x13, x13, x16      // добавляем к счётчику
    lsr x14, x14, #1      // сдвигаем байт вправо
    subs x15, x15, #1
    b.ne bit_loop

    // 3. Вставка результата в нужный байт итогового значения
    lsl x13, x13, x10
    orr x9, x9, x13

    // 4. Переход к следующему байту
    add x10, x10, #8
    subs x11, x11, #1
    b.ne byte_loop

    mov x0, x9           // возврат результата
    ldp x29, x30, [sp], #16
    ret

.section __TEXT,__cstring,cstring_literals
prompt:
    .asciz "Enter X: "
fmt_in:
    .asciz "%lld"
fmt_out:
    .asciz "Result: %lld\n"

```

Тестирование

Выполним три сценария тестов по работе программы, с ручным вычислением и работой программы.

Таблица 1. Расчёт количество единиц в числах для дальнейшей замены

Набор тестовых данных	X	Вычисления	Результат
1	3	$3/2=1$, $3\%2=1$	2
2	0	-	0
3	33	$33/2=1$ $33\%2=1$	2

```
● snctm@MacBook-Pro-Aisen fifthlab % ./lab5
Enter X: 3
Result: 2
● snctm@MacBook-Pro-Aisen fifthlab % ./lab5
Enter X: 0
Result: 0
● snctm@MacBook-Pro-Aisen fifthlab % ./lab5
Enter X: 33
Result: 2
○ snctm@MacBook-Pro-Aisen fifthlab %
```

Рис. 2. Результат тестов в программе

Выводы

В ходе выполнения лабораторной работы была успешно решена задача разработки программы на языке ассемблера ARM64 для поразрядной обработки данных согласно варианту 54: заменить каждый байт входного числа X на количество установленных единиц в этом байте.

Что было сделано:

Формализация задачи - определены типы данных (знаковые 64-битные длинные целые числа), этапы вычислений, алгоритм побайтовой обработки: извлечение байта, поразрядный подсчёт единиц, замена байта на полученное значение, сборка результат.

Разработка программы - написан код на ассемблере ARM64, включающий:

1. Выделение и освобождение стека с сохранением регистров (x29, x30)
2. Организацию ввода числа через scanf с учетом особенности Apple Silicon ABI
3. Реализация пользовательской функции transform_popcount, принимающий аргумент в x0 и возвращающей результат в x0
4. Побайтовая обработка цикл по 8 байтам, вложенный цикл по 8 битам в каждом байте
5. Поразрядный подсчёт единиц через инструкции AND, LSR, ADD
6. Сборка результата через LSL и ORR без влияния на соседние байты
7. Вывод результатов через printf с корректной передачей аргументов

Тестирование - проведено ручное тестирование для трёх наборов входных данных.

Результаты работы программы полностью совпали с теоретическими расчётами.

Итог: Все поставленные цели лабораторной работы достигнуты. Программа корректно работает на платформе ARM64 macOS, правильно обрабатывает ввод знаковых 64-битных чисел, выполняет побайтовую поразрядную обработку согласно заданию, и выводит результат в требуемом десятичном формате.